

Mastering CAN, CAN FD & CAN TP

A Complete Reference Guide
Frame Structures • Protocol Mechanics • Interview Q&A

Prepared for automotive embedded software engineering practice
(ABS/TCS ECU, UDS diagnostics, CAN/CAN-FD communication)

Table of Contents

- Part 1 — CAN 2.0 Fundamentals
- Part 2 — CAN FD (Flexible Data Rate)
- Part 3 — CAN TP / ISO-TP (ISO 15765-2)
- Part 4 — CAN Bus Practical Engineering (bit timing math, bus load, bus-off)
- Part 5 — Interview Questions & Answers (80+ questions, categorized)

Part 1 — CAN 2.0 Fundamentals

1.1 What is CAN and why it exists

Controller Area Network (CAN) is a serial, multi-master, message-based communication protocol developed by Bosch in the 1980s for automotive use. Before CAN, ECUs were wired point-to-point, which meant wiring harness complexity grew with every added ECU. CAN replaced this with a shared two-wire bus: every node listens to every message, and nodes decide for themselves whether a message is relevant to them (this is called a "broadcast" or "multicast" bus, not point-to-point addressing).

Key properties that make CAN suitable for real-time control systems like ABS/TCS:

- **Multi-master:** any node can start transmitting when the bus is idle — there's no central bus controller.
- **Message-based, not node-based addressing:** messages carry an Identifier (ID) that represents the content/meaning of the message, not the sender or receiver address.
- **Non-destructive bitwise arbitration:** if two nodes transmit at the same time, the message with the numerically lower ID wins without any data being corrupted or retransmission needed for the winner.
- **Built-in error detection** at multiple layers (bit monitoring, CRC, stuff, form, ACK checks).
- **Deterministic priority:** the lower the CAN ID, the higher the priority — critical for safety systems like ABS/TCS where wheel-speed and brake-intervention messages must win arbitration over infotainment traffic.

1.2 Physical Layer Basics

CAN uses a twisted differential pair: CAN_H and CAN_L. Two logical bus states exist:

State	Bit value	CAN_H	CAN_L	Differential (CAN_H - CAN_L)
Recessive	Logic 1	~2.5V	~2.5V	~0V
Dominant	Logic 0	~3.5V	~1.5V	~2V

Dominant always wins over recessive when driven simultaneously (this is the electrical basis for arbitration — a dominant bit physically overpowers a recessive bit on the shared bus, like a wired-AND). The bus is terminated with 120Ω resistors at each physical end (total ~60Ω differential impedance) to prevent signal reflections. High-speed CAN (ISO 11898-2) supports up to 1 Mbit/s over short distances; Low-speed/fault-tolerant CAN (ISO 11898-3) trades speed for the ability to keep working with a single-wire fault.

1.3 Frame Types — Overview

Frame Type	Purpose
Data Frame	Carries actual data from a transmitter to all receivers (most common frame)
Remote Frame	A node requests data from another node (rarely used today; DLC has no data bytes)
Error Frame	Signals that a node detected a bus error, forcing all nodes to discard the current message
Overload Frame	Requests extra delay between frames when a node needs more processing time

1.4 Standard CAN Data Frame — Full Bit-Level Structure (11-bit ID, CAN 2.0A)

This is the single most important structure to know cold. Below is every field, in transmission order, with exact bit widths.

SOF	ID[10:0]	RTR	IDE	r0	DLC	Data	CRC	CRC Del	ACK Slot	ACK Del	EOF	IFS
1 bit	11 bits	1 bit	1 bit	1 bit	4 bits	0-8 bytes	15 bits	1 bit	1 bit	1 bit	7 bits	3 bits

Field	Width	Meaning
SOF (Start of Frame)	1 bit	Always dominant (0). Signals bus-idle-to-busy transition; used by all nodes for hard synchronization.
Identifier (ID)	11 bits	Message priority/content identifier. Lower binary value = higher priority. Cannot be all-recessive in the top 7 bits combined with RTR/IDE/r0 forming a base ID all-1s pattern used by extended frames (see 1.5).
RTR (Remote Transmission)	1 bit	Dominant (0) for Data Frame, recessive (1) for Remote Frame. Also participates in arbitration — a Data Frame always wins over a Remote Frame with the same ID.

Request)		
IDE (Identifier Extension)	1 bit	Dominant (0) = 11-bit standard frame. Recessive (1) = 29-bit extended frame follows.
r0	1 bit	Reserved, always sent dominant, receivers must accept either value (forward compatibility).
DLC (Data Length Code)	4 bits	Binary value 0-8 in classic CAN, indicating number of data bytes (values 9-15 are technically encodable but classic CAN caps usable data at 8 bytes).
Data Field	0-8 bytes	Actual payload, sent MSB first, byte order defined by the application (often Motorola/big-endian or Intel/little-endian per signal, defined in the DBC).
CRC Field	15 bits	Cyclic Redundancy Check computed over SOF through end of Data field, using polynomial $x^{15}+x^{14}+x^{10}+x^8+x^7+x^4+x^3+1$.
CRC Delimiter	1 bit	Always recessive (1); separates CRC field from ACK field, and is NOT included in bit stuffing.
ACK Slot	1 bit	Transmitter sends recessive (1); any receiver that got the frame correctly overwrites it with a dominant (0) bit. This is how the transmitter knows at least one node received the frame OK — note this does NOT mean the intended application received/used it, only that CRC matched at the bit level for at least one listener.
ACK Delimiter	1 bit	Always recessive (1).
EOF (End of Frame)	7 bits	All recessive (1). Marks frame end. Deliberately NOT bit-stuffed, so 7 consecutive recessive bits here are legal and expected.
IFS (Inter-Frame Space)	3 bits	Minimum idle time (all recessive) before the next frame can start.

Important nuance: the ACK slot is a wired-OR across ALL listening nodes — the transmitter cannot tell WHICH node(s) acknowledged, only that at least one did. A single node with a CRC error will NOT ACK, but if even one other correctly-wired node on the bus does ACK, the transmitter sees a successful ACK.

1.5 Extended CAN Data Frame (29-bit ID, CAN 2.0B)

Used when more than 2048 unique priorities are needed (11-bit only gives $2^{11} = 2048$). The extended frame reuses the 11-bit field as the "Base ID," then adds an 18-bit "Extension" after IDE:

SOF 1	Base ID[10:0] 11 bits	SRR 1	IDE=1 1	Ext ID[17:0] 18 bits	RTR 1	r1 1	r0 1	DLC 4	Data 0-8B	CRC 15	...rest same as standard frame
----------	--------------------------	----------	------------	-------------------------	----------	---------	---------	----------	--------------	-----------	--------------------------------

SRR (Substitute Remote Request) sits where RTR would be in the base frame — always sent recessive so that if a standard-ID and extended-ID frame with the same base ID arbitrate together, the standard frame (which has a real RTR=0 possible) wins. This is why, at the same numerical base ID, an 11-bit standard data frame always has priority over a 29-bit extended frame.

1.6 Bit Stuffing

To guarantee enough edge transitions for receivers to stay synchronized (and to make certain error conditions detectable), CAN uses **bit stuffing**: after 5 consecutive bits of the same polarity (in the SOF through CRC field only — NOT in CRC delimiter, ACK, EOF, IFS), the transmitter inserts one bit of the opposite polarity. Receivers remove these stuff bits automatically. If a receiver sees a 6th consecutive identical bit where a stuff bit was expected, that's a **Stuff Error** — one of the five detectable CAN error types.

Worked example — where does a stuff bit get inserted?

Say the raw bits about to go on the bus (before stuffing) are: **1 0 1 1 1 1 1 0 0 1**. Scanning left to right, the transmitter counts identical consecutive bits: **1,0,1** then **1,1,1,1,1** — that's **five consecutive 1s** (positions 3-7). Right after the 5th one, the transmitter automatically inserts a stuff bit of the opposite polarity (a **0**), giving the actual bus sequence: **1 0 1 1 1 1 1 0(stuffed) 0 0 1**. The receiver, knowing this rule, silently strips that stuffed 0 back out during decoding — the application never sees it. If that stuffed 0 had somehow also come out as a 6th same-polarity bit (e.g., due to a fault), every listening node would flag a Stuff Error simultaneously.

1.7 Arbitration — Worked Example

Suppose Node A transmits ID `0x0A2` (binary `010 1000 0010`) and Node B transmits ID `0x0A5` (binary `010 1000 0101`) at exactly the same time. Both start driving bits onto the bus and simultaneously read back what's actually on the bus (this is called "monitoring while transmitting"):

- Bits 1-8 are identical (`0101000 0`) — no conflict, both keep transmitting.
- Bit 9: Node A sends 0 (dominant), Node B sends 1 (recessive) — hold on, let's check: A = ...0010, B = ...0101. Comparing bit by bit from MSB: they diverge at the bit where A has 0 and B has 1.
- Whichever node reads back a dominant bit on the bus while it transmitted recessive realizes it lost arbitration and immediately stops driving, switching to receive-only mode for the rest of that frame.
- The winning node (lower numeric ID = A, `0x0A2 < 0x0A5`) never even notices — it continues transmitting as if it owned the bus alone.

This is why CAN ID allocation in an ECU project is a **safety and real-time design decision**, not just a naming convention — the lowest IDs must go to the most time-critical, safety-relevant messages (e.g., wheel speed / brake pressure messages in ABS/TCS systems typically get very low IDs).

1.8 Error Detection Mechanisms (5 types)

Error Type	Detected By	Condition
Bit Error	Transmitter	Transmitter monitors the bus and reads back a different bit than it sent (outside arbitration/ACK fields, where this is expected).
Stuff Error	Any node	6 consecutive bits of the same polarity detected where stuffing should have broken it up.
CRC Error	Receiver	Locally computed CRC over the received frame doesn't match the transmitted CRC field.
Form Error	Any node	A fixed-format field (CRC delimiter, ACK delimiter, EOF) contains an illegal bit value.
ACK Error	Transmitter	No node dominant-ACKed the frame (ACK slot stayed recessive).

When any node detects an error, it immediately transmits an **Error Frame**: 6 dominant bits (Active Error Flag) if the node is in Error Active state, which deliberately violates bit stuffing and forces every other node to also detect an error and abandon the current frame — this is how CAN guarantees data consistency across all nodes (a frame is either accepted by everyone or by no one).

Worked example — a receive error walkthrough

Suppose Node C is receiving a frame and its locally-computed CRC doesn't match the CRC field it just received. Step by step: (1) Node C detects a **CRC Error** at the CRC-check point, right after the CRC delimiter is expected. (2) Because Node C is (assume) Error Active, it immediately transmits an **Active Error Flag** — 6 dominant bits — overriding whatever the transmitter was about to send next (the ACK slot / EOF). (3) Every other node on the bus, including the original transmitter, sees this unexpected run of dominant bits where a stuff-bit-free recessive field was expected, and each independently also detects an error (typically a Stuff or Form error, since the flag itself violates the expected bit pattern). (4) All nodes discard the frame. (5) The original transmitter, seeing this, will automatically re-attempt transmission of the same message once the bus is free again (CAN controllers do this automatically at the hardware level — this is why a single glitch is usually invisible to the application layer). (6) Node C's REC increases (it detected the error) and the original transmitter's TEC increases too (since the frame it sent was ultimately not successfully acknowledged/completed).

Worked example — TEC/REC counter progression

Assume Node D starts at TEC=0 (freshly powered up, Error Active). It has a wiring fault causing repeated Bit Errors on every transmit attempt:

Event	TEC change	Running TEC	State
Power-up	—	0	Error Active
1st failed transmit (Bit Error)	+8	8	Error Active
15 more consecutive failed transmits	+8 each	128	Crosses into Error Passive (TEC ≥ 128)
16 more consecutive failed transmits	+8 each	256	Crosses into Bus-Off (TEC > 255) — node disconnects

This shows how fast a genuinely faulty node gets quarantined — roughly 32 consecutive failed transmissions (a fraction of a second on a busy bus) is enough to go from a clean start to full Bus-Off, which is intentional: CAN is designed to fail a broken node

quickly rather than let it degrade the whole network slowly.

1.9 Error Confinement — Error Active / Passive / Bus-Off

Every CAN node maintains two internal counters: **TEC** (Transmit Error Counter) and **REC** (Receive Error Counter). These increment on errors (roughly +8 per detected error for the node reporting it, +1 for a passively-noticed error, with a more generous rule for receivers) and decrement (-1) on every successfully completed frame. The node moves between three states:

State	Condition	Behavior
Error Active	TEC < 128 and REC < 128 (normal/default state)	Node participates normally and, on error, sends an Active Error Flag (6 dominant bits) that actively destroys the bus traffic.
Error Passive	TEC ≥ 128 or REC ≥ 128 (but TEC ≤ 255)	Node still participates but on error sends a Passive Error Flag (6 recessive bits — essentially invisible unless the bus is otherwise idle), and must wait an extra time (Suspend Transmission, 8 bits) before its next transmission attempt.
Bus-Off	TEC > 255	Node disconnects itself entirely from the bus (stops transmitting/receiving) — a self-protection mechanism so a faulty node cannot keep hammering a healthy bus. Recovery requires 128 occurrences of 11 consecutive recessive bits observed on the bus (per ISO 11898-1), often after a controller reset.

Interview favorite: "Why does Bus-Off exist?" — Because without it, a node with a hardware fault (e.g., stuck-dominant transceiver or firmware bug) could otherwise keep flooding a healthy bus with error frames forever, taking every other (healthy) node down with it via cascading errors. Bus-Off is CAN's way of quarantining a misbehaving node.

1.10 Bit Timing

Each nominal bit time is divided into 4 segments, measured in Time Quanta (Tq), where Tq is a programmable fraction of the oscillator period:

Segment	Purpose	Typical length
SYNC_SEG	Fixed 1 Tq window where an edge is expected (bus transition happens here in an ideal world)	1 Tq (fixed)
PROP_SEG	Compensates for physical delays: transceiver loop delay + bus propagation delay (wire length)	1-8 Tq
PHASE_SEG1	Buffer before the sample point; can be lengthened by resynchronization	1-8 Tq
PHASE_SEG2	Buffer after the sample point; can be shortened by resynchronization	1-8 Tq, must be ≥ SJW and ≥ Information Processing Time

The **Sample Point** is the boundary between PHASE_SEG1 and PHASE_SEG2 — the instant the bit value is actually latched. It's normally placed around 75-87.5% into the bit time for classic CAN. **SJW (Synchronization Jump Width)** is the maximum number of Tq that PHASE_SEG1/2 can be adjusted per resync event to compensate for oscillator drift between nodes (hard sync at SOF, resync on every recessive-to-dominant edge afterward).

Worked baud-rate example

Suppose oscillator = 16 MHz, prescaler chosen so Tq = 62.5 ns, and you configure: SYNC_SEG=1, PROP_SEG=3, PHASE_SEG1=4, PHASE_SEG2=4 (total = 12 Tq per bit).

Bit time = 12 × 62.5 ns = 750 ns → Baud rate = 1 / 750 ns ≈ **1.333 Mbit/s**. Sample point location = (1+3+4)/12 = 66.7% into the bit.

In practice you pick Tq/segment values to land the sample point near 75-80% for classic CAN (higher tolerance to reflections, lower tolerance to oscillator drift) — this is exactly the calculation tools like PCAN's bit-timing calculator or Vector CANoe's bus configuration do for you, but you should be able to do it by hand for interviews.

1.11 Bus Load

Bus load % = (sum of bit-times of all frames transmitted per second) / (bit-time budget available per second) × 100. In practice, engineers estimate it as: for each message, (frame length in bits including stuff bits × message frequency in Hz) summed across

all messages, divided by the baud rate, $\times 100$. Keeping steady-state bus load under $\sim 70\text{-}80\%$ is a common design rule of thumb to leave headroom for burst traffic and diagnostics without risking frame queuing delays on safety-critical low-ID messages.

Worked example

Bus runs at 500 kbit/s. A wheel-speed message (standard 11-bit ID, 8 data bytes) is sent at 100 Hz. Rough frame length (worst realistic case, including typical stuffing overhead) ≈ 130 bits for an 8-byte standard frame (SOF 1 + ID 11 + RTR/IDE/r0 3 + DLC 4 + data 64 + CRC 15 + CRC-delim 1 + ACK 2 + EOF 7 + IFS 3 = 111 raw bits, plus roughly 15-20% extra from bit stuffing on the non-fixed fields ≈ 130 bits).

Bits per second for this one message = $130 \text{ bits} \times 100 \text{ Hz} = 13,000 \text{ bit/s}$. As a fraction of the 500,000 bit/s bus: $13,000 / 500,000 = \mathbf{2.6\% \text{ bus load}}$ from this single message. If you have, say, 25 similar periodic messages averaged around that size/rate, you'd be near 50-65% just from that set — which is why real ECU networks budget bus load carefully message-by-message during network design, not just check it after the fact.

Part 2 — CAN FD (Flexible Data Rate)

2.1 Why CAN FD was created

Classic CAN maxes out at 8 data bytes and effectively ~1 Mbit/s. As ECUs needed to send richer signal sets (e.g., more granular torque/wheel-speed data, larger diagnostic payloads) more often, 8 bytes became a bottleneck, forcing signal-splitting across multiple CAN IDs. CAN FD (ISO 11898-1:2015) was developed by Bosch to allow: (a) up to **64 data bytes** per frame, and (b) a faster **bit rate during the data phase** (arbitration phase stays at the classic speed for backward compatibility of arbitration logic).

2.2 CAN FD Frame Structure — What Changes

SOF	ID	RRS	IDE	FDF	res	BRS	ESI	DLC	Data	CRC	CRC Del	ACK	ACK Del	EOF
1	11/29 bit	1	1	1 (=1)	1	1	1	4	0-64B	17/21 bit	1	1	1	7

Field	Classic CAN	CAN FD
Identifier	11 or 29 bit, same	Same as classic — arbitration is identical, this is why FD stays backward compatible on the bus.
FDF (was r0)	reserved, always 0	FDF = 1 marks this as a CAN FD frame (this is the bit that tells classic-CAN-only controllers to ignore/error on it if they don't support FD — hence why a mixed bus needs all nodes FD-tolerant).
BRS (Bit Rate Switch)	N/A	If BRS=1, the bit rate switches to a faster "data phase" rate from right after BRS until the CRC delimiter. Arbitration phase (ID + control) always stays at the nominal/slower rate so arbitration integrity is preserved.
ESI (Error State Indicator)	N/A	Transmitter reports its own error state: 0 = Error Active, 1 = Error Passive. Lets receivers know the health of the sender without extra messages.
DLC	0-8 maps directly to byte count	0-8 same as classic, but 9-15 map non-linearly to larger payloads (see table below) since 4 bits can't directly count to 64.
Data field	0-8 bytes	Up to 64 bytes
CRC	15-bit, one polynomial	17-bit CRC for frames ≤16 data bytes, 21-bit CRC for frames >16 data bytes — longer payloads need stronger CRCs to keep the same error-detection probability.
Stuff bits	Simple 5-same-polarity rule	Adds a fixed stuff bit after the FDF/BRS/ESI area and a stuff-bit counter (parity-protected) inside the CRC field, to prevent long unstuffed CRC sequences from hiding errors at the higher data-phase rate.

DLC-to-byte-length mapping in CAN FD

DLC (binary)	DLC (dec)	Data Bytes
0000-1000	0-8	0-8 (1:1, same as classic)
1001	9	12
1010	10	16
1011	11	20
1100	12	24
1101	13	32
1110	14	48
1111	15	64

Memorize this table — "what data length does DLC=13 mean in CAN FD?" (answer: 32 bytes) is a very common interview trap question.

Worked example — decoding an actual CAN FD frame

Suppose a bus trace shows a CAN FD frame with ID `0x18F`, control byte (FDF/BRS/ESI/DLC nibble) indicating **BRS=1**, **ESI=0**, **DLC=1101 (13)**, and 32 data bytes follow, ending in a 21-bit CRC. Reading this off:

- **FDF=1** confirms it's an FD frame (not classic CAN, even though the ID looks ordinary).
- **BRS=1** means the data phase (from right after BRS through the CRC) ran at the faster configured data-phase bit rate, e.g. 2 Mbit/s, while the ID/arbitration portion ran at the slower nominal rate, e.g. 500 kbit/s.
- **ESI=0** tells every receiver the transmitting node currently considers itself Error Active — healthy.
- **DLC=13 (1101)** maps to **32 data bytes** per the FD DLC table (Part 2.2) — matching the 32 bytes actually observed.
- Since payload >16 bytes, the frame uses the **21-bit CRC** (not 17-bit), consistent with the CRC field length noted.

Practical takeaway: when you're staring at a raw FD trace, you can't just eyeball "8 bytes = classic, more = FD" — you decode FDF first to confirm frame type, then DLC through the FD table (not a direct number) to know the real payload size.

2.3 Bit Rate Switching (BRS) in Practice

A typical config: arbitration/nominal phase at 500 kbit/s, data phase at 2 Mbit/s or higher. The faster data-phase clock demands tighter oscillator tolerance and careful sample-point tuning because there's less time margin per bit. This is where **Transceiver Delay Compensation (TDC)** matters: at high data-phase speeds, the transceiver's own loop delay (signal round-trip through the transceiver and bus) can become a significant fraction of the bit time, so the controller applies a measured/configured compensation offset to the sample point in the data phase to avoid mis-sampling.

2.4 CAN FD and Classic CAN Coexistence

A classic-CAN-only node that doesn't understand FDF=1 will typically flag a Form Error or simply can't decode the frame, but won't corrupt the bus as long as it doesn't try to ACK/interject mid-frame — hence "CAN FD tolerant" transceivers/controllers exist for exactly this reason (they can survive an FD frame on the wire even if the application layer doesn't use FD). True mixed-mode buses require every node to be at least FD-passive-tolerant.

Part 3 — CAN TP / ISO-TP (ISO 15765-2)

3.1 Why CAN TP exists

Classic CAN frames carry only 8 data bytes, but diagnostic services (UDS, ISO 14229) routinely need to send much larger payloads — e.g., a full DTC snapshot, a security seed/key exchange, or a firmware block during ECU reprogramming can be hundreds or thousands of bytes. CAN TP (ISO 15765-2), the Transport Protocol layer, segments a large message into multiple CAN frames on the sender side and reassembles them on the receiver side, with flow control so the receiver isn't overwhelmed.

3.2 The Four N_PDU Types (PCI — Protocol Control Information)

The very first nibble of the CAN data payload is always the PCI type identifier:

PCI Type	Nibble (top 4 bits of byte 0)	Used when
Single Frame (SF)	0x0	Entire message fits in one CAN frame (≤ 7 bytes for classic CAN, more for FD — see 3.6)
First Frame (FF)	0x1	Message is larger than one frame; this is the first of several frames and announces total length
Consecutive Frame (CF)	0x2	Subsequent frames after FF, carrying the rest of the payload in order
Flow Control (FC)	0x3	Sent by the RECEIVER back to the sender, telling it whether/how to continue sending Consecutive Frames

Single Frame (SF) — byte layout (classic CAN, 8-byte frame)

PCI=0x0 nibble	SF_DL nibble, 0-7	Data byte 1..7
-------------------	----------------------	----------------

Byte 0 upper nibble = 0x0 (SF type), lower nibble = data length (0-7 for classic CAN, since 1 byte of the 8 is consumed by PCI). Remaining bytes 1-7 carry the payload, padded with 0xCC or 0x00 per the ECU's configured padding convention if the payload is shorter than 7 bytes (CAN frames are commonly padded to a fixed DLC=8 for classic CAN diagnostics per ISO 15765-2's recommendation).

First Frame (FF) — byte layout

PCI=0x1 nibble	FF_DL (bits 11:8) nibble	FF_DL (bits 7:0) byte	Data byte 1..6
-------------------	-----------------------------	--------------------------	----------------

Byte 0 upper nibble = 0x1, and the combined 12 bits (byte0 lower nibble + byte1) encode the TOTAL message length (0-4095 bytes for the standard 12-bit length field). If the total length exceeds 4095, an "escape sequence" FF_DL=0x000 is used followed by a 4-byte extended length field. The First Frame always carries exactly 6 data bytes of payload (for classic 8-byte CAN) regardless of total message size.

Consecutive Frame (CF) — byte layout

PCI=0x2 nibble	SN nibble, 0-15 rolling	Data byte 1..7
-------------------	----------------------------	----------------

Byte 0 upper nibble = 0x2, lower nibble = Sequence Number (SN), which starts at 1 for the first Consecutive Frame after the First Frame and increments mod 16 (wraps 1,2,...,15,0,1,...). The receiver uses SN to detect lost or out-of-order frames. Each CF carries 7 data bytes (classic CAN).

Flow Control (FC) — byte layout

PCI=0x3 nibble	FS nibble	BS byte	STmin byte
-------------------	--------------	------------	---------------

Field	Meaning
FS (Flow Status)	0 = CTS (Clear To Send, continue), 1 = WAIT (pause, another FC will follow), 2 = OVFLW (Overflow — abort, receiver buffer)

	can't handle the announced FF_DL)
BS (Block Size)	How many Consecutive Frames the sender may transmit before it must wait for the next Flow Control. BS=0 means "send all remaining frames without further FC" (unlimited block).
STmin (Separation Time minimum)	Minimum time the sender must wait between transmitting successive Consecutive Frames. 0x00-0x7F = 0-127 ms directly; 0xF1-0xF9 = 100-900 microseconds; other values reserved.

Worked example — a BS-limited transfer with a WAIT frame

Total payload = 25 bytes, classic CAN, tester's Flow Control this time says **BS=2, STmin=10ms** (only 2 CFs per block, then wait for another FC), and the ECU needs extra time mid-transfer:

Step	Frame	What happens
1	ECU sends FF (FF_DL=25, 6 data bytes)	Announces total length, sends first 6 bytes
2	Tester sends FC: FS=0(CTS), BS=2, STmin=10ms	"Send 2 Consecutive Frames, 10ms apart, then wait for me again"
3	ECU sends CF SN=1 (7 bytes), waits 10ms, sends CF SN=2 (7 bytes)	That's the allowed block of 2 — 6+7+7=20 bytes sent so far, ECU now stops and waits for the next FC (per BS=2 rule)
4	Tester needs a moment (e.g. its own buffer housekeeping) and sends FC: FS=1 (WAIT)	Tells the ECU "don't send yet, I'll send another FC soon" — transfer is paused, not aborted
5	Tester sends FC: FS=0(CTS), BS=0, STmin=10ms	Now clears the ECU to send the rest (5 remaining bytes) without further limit
6	ECU sends CF SN=3 (final 5 bytes, padded)	6+7+7+5=25 bytes total — transfer complete

This shows exactly how BS chops a large transfer into receiver-paced chunks, and how a WAIT frame can insert a pause anywhere in that sequence without killing the whole transfer — as long as it arrives before the sender's N_Bs timeout expires.

Worked example — extended addressing

With extended addressing, CAN ID 0x7E0 might be shared by several logical sub-functions, distinguished by an address-extension byte placed as the very first data byte, before the PCI. E.g., data bytes 05 02 10 03 : byte 0 = 0x05 is the target address extension (says "this message is for sub-function 5"), byte 1 = 0x02 is now the actual PCI (Single Frame, SF_DL=2), and bytes 2-3 = 10 03 are the 2 payload bytes (a UDS DiagnosticSessionControl request to session 0x03, for example). Compare this to normal addressing, where byte 0 itself would directly be the PCI (02 10 03) — extended addressing costs exactly 1 byte of usable payload per frame in exchange for finer-grained addressing under one shared CAN ID.

3.3 Addressing Modes

Mode	How address info is carried	Notes
Normal addressing	Entirely in the CAN ID (e.g., request ID 0x7E0, response ID 0x7E8 — classic UDS physical pair)	All 8 (or up to 64 for FD) data bytes available for PCI+payload
Extended addressing	CAN ID + first data byte is a target address extension	1 byte of payload capacity is "spent" on addressing
Mixed addressing	CAN ID + first data byte used as address extension in a specific hybrid scheme	Used in some OEM-specific/functional addressing schemes

Separately, UDS/CAN TP also distinguishes **Physical addressing** (1-to-1: tester talks to exactly one ECU, e.g., request 0x7E0/response 0x7E8) from **Functional addressing** (1-to-many: a broadcast request like 0x7DF that many ECUs may listen to and respond to individually, typically only usable with Single Frame requests since multiple ECUs can't coordinate a shared Flow Control response).

3.4 Timing Parameters (N-Layer Timers)

Timer	Measured where	Meaning
N_As	Sender	Time allowed for the CAN layer to accept/transmit a frame the TP layer handed it (sender side "As" = sending)
N_Ar	Receiver	Same, but for the receiver sending a Flow Control frame back
N_Bs	Sender	Max time the sender will wait for a Flow Control frame from the receiver after sending FF or after a BS-limited block of CFs

N_Cr	Receiver	Max time the receiver will wait for the next Consecutive Frame from the sender
N-Cs	Sender	(Not standardized as a hard timeout, but) the sender's own internal cadence honoring STmin between CFs

Typical default value in ISO 15765-2 for N_Bs and N_Cr timeouts is 1000 ms; exceeding them aborts the transfer.

3.5 Worked Example — Full Multi-Frame Transfer

Say a UDS response payload is 20 bytes long, sent over classic 8-byte CAN, normal addressing, response ID 0x7E8, with the tester's Flow Control saying BS=0 (send all), STmin=0.

Direction	CAN Bytes (hex)	Explanation
ECU → Tester (FF)	10 14 62 F1 90 31 32	PCI=0x1 (FF), FF_DL=0x014=20 total bytes, then first 6 payload bytes (62 F1 90 31 32... — this example shows a ReadDataByIdentifier positive response starting)
Tester → ECU (FC)	30 00 00	PCI=0x3 (FC), FS=0 (CTS), BS=0x00 (send all remaining), STmin=0x00 (no minimum delay)
ECU → Tester (CF, SN=1)	21 33 34 35 36 37 38 39	PCI=0x2, SN=1, next 7 payload bytes
ECU → Tester (CF, SN=2)	22 30 31 32 33 34 35 36	PCI=0x2, SN=2, next 7 payload bytes (6+7+7=20, transfer complete)

Total payload reassembled by tester = 20 bytes = the 5 bytes from FF (after PCI+length, minus the 2 length bytes = 6 actual data bytes) + 7 + 7 = 20. Transfer complete once the receiver has collected FF_DL total bytes.

3.6 CAN FD TP Differences (ISO 15765-2 for CAN FD)

- Since CAN FD frames can carry up to 64 bytes, a Single Frame can carry far more payload — up to 62 bytes when a 2-byte PCI length is used for lengths >62 (an "escape" SF_DL=0x00 followed by a full length byte), or up to 63 bytes with the short form on FD frames using a full DLC=64 encoding.
- First Frame and Consecutive Frame payload capacity scales with the FD DLC used (e.g., a First Frame on a 64-byte FD frame carries far more than 6 bytes per frame), dramatically reducing the number of frames (and thus time) needed for large transfers such as ECU reprogramming block downloads — directly relevant to flashing tools like yours.
- Padding conventions and the CAN FD DLC-to-length non-linear table (Part 2.2) must be respected when a TP layer picks how many bytes to actually put in a frame — you don't get to pick "37 bytes," you must round to a legal FD DLC length (e.g., 48).

Part 4 — Practical Engineering Notes

4.1 Reading a Raw Bus Trace

A typical `candump` / PCAN-View log line looks like:

```
(1692000001.234567) can0 0C2#0A1B2C3D4E5F6071
```

This decodes as: timestamp, interface, CAN ID `0x0C2` (standard frame, since no extended-ID marker), then 8 data bytes in hex. Cross-referencing the ID against a DBC file lets you decode individual signals (bit position, length, scaling, offset, endianness) out of that byte string — a skill worth practicing manually before relying on tools like CANoe/BUSMASTER.

Worked example — manually decoding one signal from raw bytes

Suppose the DBC says: for ID `0x0C2`, signal `WheelSpeed_FL` starts at bit 0 of byte 0, is 16 bits long, little-endian (Intel byte order), factor = 0.01, offset = 0 (i.e., raw $\times$ 0.01 = km/h).

From the trace above, data bytes are `0A 1B 2C 3D 4E 5F 60 71`. Little-endian means the first two bytes (byte 0, byte 1) combine as **low byte first**: raw value = $\text{byte1} \ll 8 \mid \text{byte0} = 0x1B \ll 8 \mid 0x0A = 0x1B0A = 6922$ decimal.

Apply the scaling: $6922 \times 0.01 = \mathbf{69.22 \text{ km/h}}$. That's the actual physical wheel speed value your application would use — this exact raw-bytes-to-physical-value process is what your DBC-decoding logic (or CANoe/BUSMASTER's panel) does automatically, but you should be able to trace it by hand when debugging a signal that "looks wrong" on a tool, since tool misconfiguration (wrong endianness, wrong factor) is a common real-world bug source.

4.2 Common Field Issues and Root Causes

Symptom	Likely Cause
Node goes Bus-Off repeatedly	Wiring fault (open/short on CAN_H or CAN_L), missing/duplicate 120 Ω termination, ground offset between ECUs, or a node transmitting with wrong bit-timing config vs the rest of the bus
Intermittent CRC errors only at high bus load	Marginal bit-timing/sample-point margin, EMI susceptibility, or a node with a slightly drifting oscillator lacking proper resync
UDS multi-frame transfer times out (N_Bs exceeded)	Tester or ECU not responding to Flow Control fast enough, wrong CAN ID pairing, or STmin/BS mismatch between tester tool config and ECU capability
One ECU never ACKs	ECU not powered/not on bus, wrong baud rate configured on that node, or it filtered out the ID entirely at the acceptance filter

Part 5 — Interview Questions & Answers

80+ questions organized by category, ranging from fundamentals to the kind of scenario questions asked of Tier-1 automotive embedded engineers with diagnostic/flashing experience.

5.1 CAN Fundamentals

Q1. What is the difference between dominant and recessive bits, and why does dominant always win?

Dominant = logic 0, driven actively by transistors pulling CAN_H high and CAN_L low, creating a large differential. Recessive = logic 1, the bus's passive/idle state maintained by termination resistors. Because dominant is actively driven and recessive is passive, if one node drives dominant while another drives recessive, the bus physically reads dominant — a wired-AND effect. This is the electrical basis of non-destructive arbitration.

Q2. Why is a lower CAN ID higher priority?

During arbitration, nodes compare their ID bit-by-bit against what's actually on the bus. A 0 (dominant) bit in the ID will "beat" a 1 (recessive) bit at the same position, because the bus goes dominant regardless of what the recessive-sender intended. So a numerically smaller ID (more leading zeros) is statistically more likely to stay dominant longer and win.

Q3. What happens when two nodes with the same CAN ID transmit simultaneously?

This is a bus design fault (IDs should be unique). In practice, since IDs are identical, they wouldn't diverge during arbitration, and if their data content also differs, this leads to bit errors post-arbitration when the transmitters read back a bus value that doesn't match what they sent, which triggers error frames. Some systems intentionally use this for particular redundancy schemes, but generally it's avoided by design.

Q4. Explain bit stuffing and why it's needed.

After 5 consecutive same-polarity bits, a stuff bit of opposite polarity is inserted (in SOF through CRC field only). This guarantees enough signal transitions for receivers' clocks to resynchronize, and makes 6-in-a-row same-polarity bits an unambiguous error condition (Stuff Error).

Q5. What are the 5 CAN error types?

Bit Error, Stuff Error, CRC Error, Form Error, ACK Error (see Part 1.8 for full definitions and who detects each).

Q6. Explain the three error states of a CAN node and what triggers Bus-Off.

Error Active (TEC/REC < 128, normal operation), Error Passive (TEC or REC ≥ 128, node becomes "quieter" on errors and must wait extra time before transmitting), and Bus-Off (TEC > 255, node disconnects entirely). Bus-Off protects a healthy bus from a persistently faulty node.

Q7. How does a node recover from Bus-Off?

Per ISO 11898-1, the node must observe 128 occurrences of 11 consecutive recessive bits on the bus before re-entering Error Active state (many microcontroller CAN peripherals implement this automatically once re-enabled, but some require an explicit reset/reinit sequence, so it's implementation-dependent — worth checking your specific MCU's CAN/FDCAN peripheral reference manual).

Q8. What's the maximum data payload in a classic CAN frame, and why?

8 bytes, because the DLC field is 4 bits (0-15 possible) but classic CAN only defines 0-8 as valid lengths; this was a deliberate 1980s tradeoff between frame overhead and RAM/latency constraints of that era's silicon.

Q9. What is the difference between a Data Frame and a Remote Frame?

A Data Frame carries the RTR bit dominant (0) and carries actual payload. A Remote Frame carries RTR recessive (1) and has no data bytes — it's a request for the node owning that ID to respond with a Data Frame. Remote frames are rarely used in modern designs; most OEMs use cyclic/event-based Data Frames instead.

Q10. What's the difference between the standard (11-bit) and extended (29-bit) CAN identifier frames?

11-bit gives 2048 unique IDs; 29-bit gives ~536 million. Extended frames reuse the 11-bit Base ID field plus an 18-bit extension, with an SRR bit standing in for RTR so that, at equal Base ID, standard frames always win arbitration over extended frames.

Q11. Why is the CRC delimiter, ACK delimiter, and EOF not bit-stuffed?

These are fixed-format fields with a known, expected bit pattern (all recessive). Bit stuffing there would make frame length/timing unpredictable and isn't necessary since these fields don't need extra edges for resync — plus it allows the fixed pattern itself to be used for Form Error detection.

Q12. What is the ACK slot and what does a successful ACK actually guarantee?

It's 1 bit where the transmitter sends recessive, expecting at least one properly-receiving node to overwrite it with dominant. A successful ACK only guarantees that at least one node on the bus received the frame with no CRC/form/stuff errors at the bit level — it does NOT guarantee the "correct" or "intended" receiver got it, nor that the application layer processed it.

Q13. What is the purpose of bus termination resistors, and what happens without them?

120Ω resistors at each physical bus end absorb the signal and prevent reflections that would otherwise corrupt bit edges, especially at higher baud rates and longer bus lengths. Without proper termination (missing, wrong value, or too many resistors), you typically get a degraded differential signal, increased susceptibility to noise, and often intermittent CRC/stuff errors or an inability to communicate at all at higher speeds.

Q14. What's the difference between Non-Return-to-Zero (NRZ) encoding used by CAN and other line codes like Manchester?

CAN uses NRZ: a bit's voltage level is held for the entire bit time with no forced mid-bit transition (unlike Manchester, which transitions every bit). This is more bandwidth-efficient but requires bit stuffing to guarantee enough transitions for clock recovery, since NRZ alone can have long runs with no edges.

Q15. How is the CAN bit time divided, and what is the Sample Point?

Into SYNC_SEG, PROP_SEG, PHASE_SEG1, PHASE_SEG2 (measured in time quanta). The Sample Point is the boundary between PHASE_SEG1 and PHASE_SEG2 where the bit's actual logical value is latched; it is typically placed at 75-87.5% of the bit time for classic CAN to allow maximum settling time for reflections while still leaving margin for oscillator drift correction.

Q16. What is SJW (Synchronization Jump Width)?

The maximum number of time quanta the controller is allowed to lengthen PHASE_SEG1 or shorten PHASE_SEG2 by, per resynchronization event, to correct for oscillator drift between the transmitting and receiving nodes. Larger SJW gives more drift tolerance but less design margin elsewhere.

5.2 CAN FD

Q17. What are the two headline improvements CAN FD offers over classic CAN?

(1) Larger payload — up to 64 bytes vs 8. (2) Higher data-phase bit rate via Bit Rate Switch (BRS) — arbitration stays at the classic/nominal rate for compatibility, but the data phase can run several times faster.

Q18. What do the FDF, BRS, and ESI bits mean?

FDF (replaces classic r0): 1 = this is an FD frame. BRS: 1 = switch to the faster data-phase bit rate for the data+CRC portion. ESI: transmitter's own error state, 0=Active, 1=Passive — lets receivers see sender health without a separate message.

Q19. Why does CAN FD use two different CRC lengths (17-bit and 21-bit)?

17-bit CRC is used for frames with data length ≤16 bytes; 21-bit CRC for >16 bytes. Longer payloads statistically need a stronger (longer) CRC polynomial to maintain the same undetected-error probability as classic CAN's 15-bit CRC did for 8-byte payloads.

Q20. What data length does DLC=1011 (11) correspond to in CAN FD?

20 bytes (see the DLC mapping table in Part 2.2 — 9→12, 10→16, 11→20, 12→24, 13→32, 14→48, 15→64).

Q21. Why does arbitration in CAN FD still run at the classic/nominal bit rate?

Arbitration relies on every node reading back the bus state within the same bit and reacting fast enough to detect a lost-arbitration condition. Running arbitration at very high speed across a bus with real propagation delay risks nodes not being able to react in time, corrupting the arbitration mechanism itself. Keeping arbitration at the proven classic rate preserves determinism and backward compatibility, while only the data phase — which doesn't need multi-node bitwise consensus — gets sped up.

Q22. What is Transceiver Delay Compensation (TDC) and why does CAN FD need it?

At high data-phase bit rates, the transceiver's own signal loop delay becomes a significant fraction of the bit time, which can shift where the "real" bit edge appears relative to the ideal sample point. TDC applies a measured or configured offset to the data-phase sample point to compensate for this delay, preserving reliable sampling at high speed.

Q23. Can a classic-CAN-only node coexist on the same bus as CAN FD nodes?

Only if it's at least "FD-tolerant" (also called FD-passive) at the transceiver/controller level — meaning it can electrically survive an FD frame on the wire without corrupting the bus, even though it can't decode FD payloads itself. A fully non-tolerant classic CAN controller will typically flag persistent errors when it sees FD frames, potentially disrupting the bus.

Q24. What additional stuffing feature does CAN FD add beyond classic bit stuffing?

A fixed stuff bit right after the FDF/res/BRS/ESI area (regardless of polarity run-length) and a stuff-bit counter with parity, included to protect the CRC field's integrity — since at high data-phase speed, a long unstuffed run in the CRC could otherwise hide certain error patterns that the extra counter helps catch.

5.3 CAN TP / ISO-TP

Q25. Why is a Transport Protocol layer needed on top of CAN for diagnostics?

Classic CAN frames only carry 8 bytes, but UDS services (e.g., reading a full DTC snapshot, security seed/key exchange, ECU flash block transfer) often need much larger payloads. CAN TP (ISO 15765-2) segments large payloads into multiple frames and reassembles them on the receive side, with flow control to pace the transfer.

Q26. List the four N_PDU (PCI) types and their identifying nibble.

Single Frame = 0x0, First Frame = 0x1, Consecutive Frame = 0x2, Flow Control = 0x3 — always the upper nibble of the first payload byte.

Q27. How much payload can a Single Frame carry on classic 8-byte CAN, and why?

Up to 7 bytes, because 1 byte's upper nibble is used for the PCI type and its lower nibble for the length (SF_DL, 0-7).

Q28. Walk through how a First Frame announces total message length.

Byte 0's lower nibble plus all of byte 1 together form a 12-bit FF_DL field (0-4095 bytes). If the message exceeds 4095 bytes, an escape sequence (FF_DL=0x000 in the 12-bit field) is used, followed by a separate 4-byte extended length field for messages up to ~4GB.

Q29. What does the Sequence Number (SN) in a Consecutive Frame do, and how does it wrap?

It's a 4-bit rolling counter starting at 1 for the first CF after the FF, incrementing by 1 for each subsequent CF, wrapping from 15 back to 0 (not back to 1). The receiver uses it to detect missing, duplicated, or reordered frames and can raise a Wrong Sequence Number error if it doesn't match expectation.

Q30. Explain the three Flow Status (FS) values in a Flow Control frame.

0 = CTS (Clear To Send — proceed with the next block of Consecutive Frames), 1 = WAIT (pause; sender must wait for a further FC before continuing — used when the receiver needs more time, e.g., buffer/erase operations during flashing), 2 = OVFLW (Overflow — abort the transfer; the receiver's buffer can't accommodate the announced FF_DL).

Q31. What do BS and STmin control, and what does BS=0 mean?

BS (Block Size) caps how many Consecutive Frames the sender may transmit before waiting for another Flow Control; BS=0 means "no limit — send the entire rest of the message without further FC." STmin (Separation Time minimum) sets the minimum gap the sender must leave between consecutive CF transmissions, to avoid overrunning a slower receiver's buffer.

Q32. What's the difference between physical and functional addressing in UDS/CAN TP?

Physical addressing is a 1-to-1 request/response pair to a specific ECU (e.g., request 0x7E0 → response 0x7E8). Functional addressing is a 1-to-many broadcast (e.g., 0x7DF) that multiple ECUs may all receive and individually respond to — typically restricted to Single Frame requests, since a multi-frame exchange needs a coordinated one-to-one Flow Control handshake that doesn't work cleanly with multiple simultaneous responders.

Q33. What happens if the sender doesn't receive a Flow Control frame within the expected time?

The N_Bs timer (sender-side wait-for-FC timeout, default 1000 ms per ISO 15765-2) expires, and the sender aborts the transfer, typically reporting a transport-layer timeout up to the application/UDS layer.

Q34. In an ECU reflashing scenario, why might a tester see repeated WAIT (FS=1) Flow Control frames?

The ECU (receiver of the flash data) may be busy internally — e.g., erasing a flash sector or writing a block to non-volatile memory — and uses WAIT frames to hold off the tester from sending more Consecutive Frames until it's ready, without aborting the whole transfer.

Q35. How does CAN FD change First Frame and Consecutive Frame payload capacity?

Since CAN FD frames can carry up to 64 bytes instead of 8, both FF and CF can carry proportionally more payload bytes per frame (bounded by the legal FD DLC-to-length table), meaning far fewer frames — and less time — are needed to transfer the same total payload, which matters directly for flash-block-transfer speed during reprogramming.

Q36. What's the difference between extended addressing and normal addressing in CAN TP, and what's the tradeoff?

Normal addressing uses only the CAN ID for addressing, leaving the full data field for PCI+payload. Extended addressing reserves the first data byte as an address extension (on top of the CAN ID), which costs 1 byte of usable payload capacity per frame but allows more granular target addressing when the CAN ID space alone isn't descriptive enough.

5.4 Scenario / Troubleshooting Questions

Q37. Your ECU intermittently goes Bus-Off only under high bus load. What would you check first?

Bit-timing/sample-point margin (is the sample point too early/late for the wiring length and node count?), oscillator tolerance and SJW sufficiency, physical layer health (termination, connector/ground issues) that only manifests as noise-sensitive errors under heavier traffic, and whether TEC is climbing due to genuine bit/CRC errors versus a marginal design that only fails under worst-case bus loading and simultaneous transmitter contention.

Q38. A UDS multi-frame download to an ECU keeps timing out at N_Bs. What are the possible causes and how do you isolate them?

Possible causes: ECU not sending Flow Control at all (check ECU firmware TP stack), Flow Control sent on the wrong CAN ID (physical addressing mismatch), tester tool's N_Bs timeout configured too aggressively for how long the ECU legitimately needs, or a bus-level issue (wiring/termination) dropping the FC frame outright. Isolate with a bus trace (candump/CANoe) captured during the failure to see whether an FC frame appears on the wire at all, and if so, whether it's correctly addressed and timed.

Q39. Two ECUs on the bus have the same CAN ID for different signals by mistake. What symptoms would you expect?

Depending on data content and timing, you'd likely see intermittent Bit Errors (when both drive the bus with actually different data mid-frame, since arbitration alone can't fully resolve it if bits coincidentally match through arbitration then diverge in the data field) — this typically manifests as sporadic CRC/Bit errors and possibly rising TEC/REC on both nodes, hard to reproduce consistently, and is exactly why CAN ID uniqueness is enforced at the DBC/network design stage.

Q40. How would you calculate whether adding a new periodic 100-byte message at 50 Hz will overload a 500 kbit/s classic CAN bus that's already at 60% load?

Since classic CAN payload is capped at 8 bytes/frame, 100 bytes needs to be split via CAN TP into multiple frames (13 frames: 1 FF + 12 CF for a 100-byte payload, roughly). Estimate total bits per full logical message (each physical frame's overhead ~40-50 bits plus 8 data bytes plus stuffing, per frame) × number of frames × 50 Hz, divide by 500,000 bit/s, and add that percentage to the existing 60% to see if you cross your bus-load design margin (commonly 70-80%) — if so, you'd reconsider frequency, payload splitting strategy, or ID/priority placement.

Q41. What could cause a CAN node to never see any ACK on the bus even though it appears to transmit fine?

No other node is actually on/listening on the bus (single-node testing without a second live node or without a bus analyzer configured to ACK), all other nodes have the wrong baud rate configured, all other nodes filter out that specific ID at their acceptance filters, or a physical layer fault preventing any other node from correctly receiving the frame at the bit level.

Q42. During flashing over CAN FD, the tool reports inconsistent throughput even though BRS is enabled. What would you investigate?

Data-phase sample point and TDC configuration (is the higher-speed data phase actually stable, or intermittently erroring and forcing retransmissions/renegotiation?), STmin value being larger than necessary (unnecessarily throttling throughput), Block Size causing frequent FC round-trips, and whether the ECU is issuing WAIT frames due to internal flash-write timing that's masquerading as a "CAN" throughput problem when it's actually an ECU-side bottleneck.

5.5 Quick-Fire / Conceptual Traps

Q43. Does a successful CAN ACK mean the application layer received and correctly interpreted the message?

No — ACK only confirms bit-level reception (CRC/stuff/form correctness) by at least one node; it says nothing about whether the intended application actually accepted/processed the message's contents.

Q44. Is CAN a "point-to-point" or "broadcast" protocol?

Broadcast/multicast — every message is transmitted onto a shared bus that all nodes physically receive; each node's acceptance filter decides locally whether to keep or discard it. There's no addressed "point-to-point" delivery at the CAN data-link layer itself.

Q45. Does CAN FD's BRS change the arbitration outcome or priority scheme in any way?

No — arbitration (the ID comparison) always happens at the nominal/slower rate before BRS takes effect, so message priority ordering is completely unaffected by CAN FD's speed features.

Q46. In CAN TP, can a Flow Control frame be sent for a Single Frame message?

No — Flow Control only applies to multi-frame (First Frame/Consecutive Frame) exchanges. A Single Frame is self-contained and needs no flow-control handshake.

Q47. What's the minimum and maximum theoretical STmin, and what do the two numeric ranges represent?

0x00-0x7F represents 0-127 milliseconds; 0xF1-0xF9 represents 100-900 microseconds (sub-millisecond range) for finer-grained pacing on faster links; values 0x80-0xF0 and 0xFA-0xFF are reserved/not used.

Q48. Why can't classic CAN and CAN FD data-phase bit rates be arbitrary — why must they follow specific DLC/length rules?

Because both ends (transmitter and every receiver) must agree in advance, via the protocol's fixed encoding rules (DLC table for FD, fixed 8-byte cap for classic), on exactly how many bits/bytes constitute the frame — there's no separate "length" negotiation at the CAN data-link layer itself, so the DLC field's meaning must be a fixed, universally-known mapping.

— End of Guide —